

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 02 · LECTURE NOTES

Text as Data

Tokenisation, and why it is not a solved detail.

MODULE OUTCOMES

Explain why tokenisation is a modelling decision.

Explain subword algorithms and what they buy.

Explain the failure modes tokenisation causes.

Explain normalisation and what it destroys.

Inspect a tokeniser's actual output.

1 The problem

WHITESPACE IS NOT A WORD BOUNDARY, AND A WORD IS NOT A UNIT OF MEANING

Chinese and Japanese are written without spaces between words, so whitespace splitting produces a single enormous token — and **German compounding and Turkish agglutination produce single orthographic 'words' carrying what English would express in a clause.**

And even in English the notion does not hold together: 'New York' is one concept written as two words; 'don't' is two words written as one; 'state-of-the-art' is anyone's guess; and a URL or a chemical name is not a word at all.

So the question 'what is a word' has no language-independent answer, and arguably no answer within a single language either — which means **tokenisation is a modelling decision with downstream consequences** rather than a preprocessing detail to be handled by a utility function.

And it is the decision made earliest and revisited least. Every model in this course inherits its tokeniser's limitations — and §3 covers what those turn out to be, which is considerably more consequential than the topic's usual treatment suggests.

Unit	The advantage	The cost
Characters	No unknown tokens, ever; a vocabulary of a few hundred.	Very long sequences, and very little meaning per unit — the model must compose everything from scratch.
Words	The units carry meaning directly.	An enormous vocabulary, and unknown tokens guaranteed at inference (Module 01 §1).

Unit	The advantage	The cost
Subwords	No unknown tokens, a bounded vocabulary, and frequent words kept intact.	The splits are statistical rather than morphological (§2), which produces §3's failures.
Bytes	Universal — handles any script, any encoding, any input.	The longest sequences, and no linguistic alignment at all.

Subwords won because they bound the vocabulary *and* eliminate unknown tokens — which were previously a trade-off between the first two rows, and getting both at once is a genuine advance rather than a compromise. The cost is that the splits do not respect morphology, which is §3's subject and is a real price rather than a theoretical one.

2 Subword algorithms

TRAINING

1. start with the vocabulary of single characters
2. count every adjacent pair in the corpus
3. merge the most frequent pair into one token
4. record the merge; repeat to the target size

INFERENCE

apply the recorded merges, in the same order

S0: frequent sequences become single tokens, and rare ones remain split into pieces. "the" is one token; an unusual surname is five.

THE VARIANTS

WordPiece merges by likelihood gain rather than by raw frequency

Unigram starts from a large vocabulary and prunes, keeping the most probable segmentation of the corpus

SentencePiece treats the input as raw bytes, so no language-specific pre-tokenisation step is needed at all

AND FREQUENCY IS THE ONLY CRITERION -- which is section 3's whole problem.

Frequency is the only criterion, in every variant — so the segmentation reflects the training corpus rather than the language's structure. A morpheme that is linguistically fundamental but rare in the corpus gets split; a meaningless character sequence that happens to be frequent becomes a single token. This single property produces every failure in §3, which is why it is worth stating explicitly rather than treating the algorithms as interchangeable implementation details.

3 What tokenisation breaks

- **Arithmetic. Numbers are split inconsistently** — '1234' may be one token or three depending on how often that exact string appeared — **which makes digit-level reasoning hard for reasons that have nothing to do with arithmetic** and everything to do with whether the digits are separately represented at all.
- **Character-level tasks. A model that never sees individual letters struggles to count them, reverse a word, or reason about spelling** — **which looks like a reasoning failure and is a representation failure**, and the distinction matters for what you do about it.
- **Low-resource languages. A tokeniser trained predominantly on English splits other scripts into several times more tokens for the same content** — so the same meaning consumes more of a fixed context window and costs more money per request (Module 12 §2). **This is the clearest equity consequence in the module.**
- **Spelling and morphology** — morphologically related words receive unrelated token sequences, so **the model must learn their relationship from scratch from co-occurrence** rather than getting it from the representation.
- **And prompt sensitivity** — **a trailing space or a different quotation mark changes the tokenisation and therefore the output**, which accounts for a surprising share of the apparent brittleness people attribute to model capriciousness.

WHICH MEANS SOME APPARENT REASONING FAILURES ARE TOKENISATION FAILURES

When a model fails at counting characters, comparing numbers, or manipulating spelling, check the tokenisation before concluding anything at all about its reasoning.

Because the model may genuinely not have access to the information. If '1234' is a single opaque token, the individual digits are not separately represented — asking it to compare that against '987' is asking it to compare two arbitrary symbols, which is a task nobody could do.

And the fix is frequently a representation change rather than a model change — splitting digits individually at tokenisation time, or restating the task so the required units are visible, both of which cost nothing.

Which is a specific instance of a general habit: before attributing a failure to the model's capability, check whether the input representation made the task possible at all (Module 13 §1) — and this habit transfers well beyond language.

4 Normalisation

- **Lowercasing** loses the distinction between 'Apple' and 'apple', 'US' and 'us', 'May' and 'may' — **which matters a great deal for entity recognition and barely at all for topic classification**, and that is the shape of every decision in this list.

- **Unicode normalisation** is nearly always correct, since visually identical strings ought to be the same thing — and **it is the step most often omitted**, which produces silent duplicate vocabulary entries and occasionally a security issue (CSCE 713 Module 04 §2's encoding confusion).
- **Stripping punctuation** loses sentence boundaries, negation cues, and the difference between a statement and a question — all of which were load-bearing for the compositional tasks of Module 01 §4.
- **Stemming and lemmatisation** conflate inflected forms, which **helps a sparse count-based model with limited data and is unnecessary for a model with learned subword embeddings** — the embeddings already place the forms near one another.
- **And removing stopwords** destroys exactly the function words that carry the syntax — **appropriate for retrieval, where they contribute little and cost index space (CSCE 670), and actively harmful for anything compositional. Every step is lossy, and the right choice depends on the task** — which is why the classical preprocessing recipe is wrong for neural models and was entirely right for the models it was designed for.

THE PRACTICAL INSTRUCTION: READ YOUR TOKENISER'S OUTPUT

Print the tokens for twenty inputs drawn from your actual data — including the longest, the shortest, the numeric, the non-English, the badly formatted, and the one with your domain's key terminology in it.

You will find something surprising. A common word in your domain split into six pieces, numbers tokenised inconsistently, an emoji consuming four tokens, whitespace handled differently than you assumed, or your product's name fragmented — and any of those changes what you should do.

And the finding changes real decisions: sequence length budgets, cost estimates per request, whether a domain-specific vocabulary is worth training, and whether a task is even expressible in this representation (§3).

Which is five minutes of work that essentially nobody does — and it is the cheapest diagnostic in this entire course. Project 2 requires it, for exactly that reason.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Jurafsky & Martin, chapter 2 (free)	The whole module , with the normalisation decisions and the regular-expression machinery.
Sennrich et al. — Neural Machine Translation of Rare Words with Subword Units (free)	§2's algorithm in the original — short, and the motivation is stated plainly.
Kudo & Richardson — SentencePiece (free)	§2's variants , and the argument for removing language-specific pre-tokenisation entirely.

Resource	What it covers for this module
The Hugging Face tokenizers documentation (free)	§4's practical instruction , with the inspection API — use it on your own data today.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Tokenise the same sentence** by character, word, subword, and byte, and compare the lengths.
- 2 **Implement BPE training** on a small corpus, and print the first fifty merges.
- 3 **Apply your merges** to held-out text and confirm there are no unknown tokens.
- 4 **Tokenise fifty numbers** and report how inconsistent the splitting is.
- 5 **Tokenise the same paragraph in English and in two other languages**, and compare the token counts.
- 6 **Find a word in your domain** that your tokeniser splits badly.
- 7 **Add and remove a trailing space** from a prompt and compare the tokenisation.
- 8 **Take a task a model fails** and determine whether the tokenisation made it possible.
- 9 **Apply the five normalisation steps** to one corpus and measure the vocabulary size after each.
- 10 **Read your tokeniser's output on twenty real inputs** and write down what surprised you.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is whitespace not a word boundary, and why is a word not a unit of meaning?
- 2 Give four tokenisation granularities with their advantage and cost.
- 3 Why did subwords win?
- 4 Describe BPE training and inference.
- 5 What is the only criterion, and what follows from that?
- 6 Give five failures traceable to tokenisation.
- 7 Which has the clearest equity consequence, and why?
- 8 What should you check before concluding a reasoning failure?
- 9 Give five normalisation steps and what each destroys.
- 10 What is the cheapest diagnostic in the course?